

PHẦN 8.5 - STACK<T> TRONG C#

Nguyên tắc LIFO, thao tác an toàn, đóng gói, ứng dụng và bài tập thực hành

Mục tiêu: Hiểu cách Stack hoạt động theo LIFO; sử dụng Push, Pop, Peek, TryPop, TryPeek; xử lý nullable; đóng gói Stack trong class và áp dụng vào lịch sử thao tác.

Tài liệu này giữ đầy đủ toàn bộ nội dung của bài học vừa trình bày và không kèm đáp án cho bài tập cuối phần.

1. Stack<T> là gì?

Bạn vừa học Queue<T> theo nguyên tắc FIFO - vào trước, ra trước. Stack<T> hoạt động theo nguyên tắc ngược lại: LIFO - vào sau, ra trước.

```
Queue<T>
→ FIFO
→ vào trước, ra trước

Stack<T>
→ LIFO
→ vào sau, ra trước
```

Stack<T> được gọi là ngăn xếp.

2. Nguyên tắc LIFO

LIFO là viết tắt của Last In, First Out - phần tử được đưa vào sau cùng sẽ được lấy ra trước tiên.

```
Đĩa C ← lấy ra trước
Đĩa B
Đĩa A ← đặt vào đầu tiên
```

Nếu lần lượt thêm A, B, C thì thứ tự lấy ra là C, B, A.

```
Queue: A → B → C
Lấy ra: A trước

Stack:
C
B
A

Lấy ra: C trước
```

3. Khai báo Stack<T>

```
Stack<T> stack = new Stack<T>();
```

Ví dụ với chuỗi và object:

```
Stack<string> pages =
    new Stack<string>();

Stack<Operation> operations =
    new Stack<Operation>();
```

Trong đó T là kiểu dữ liệu được lưu trong Stack.

4. Thêm phần tử bằng Push()

```
pages.Push("Home");
pages.Push("Products");
pages.Push("Product Details");

Product Details ← đỉnh Stack
Products
Home
```

Phần tử được thêm sau cùng nằm trên đỉnh Stack. Push() có kiểu trả về void, nghĩa là không trả về giá trị.

```
pages.Push("Home"); // kiểu trả về: void
```

5. Lấy và xóa phần tử bằng Pop()

```
string page = pages.Pop();
```

Kết quả: page nhận "Product Details" và phần tử này bị xóa khỏi Stack.

```
Products
Home
```

Pop() thực hiện hai việc:

1. Lấy phần tử trên cùng.
2. Xóa phần tử đó khỏi Stack.

```
Console.WriteLine(pages.Pop());
Console.WriteLine(pages.Pop());
```

Kết quả lần lượt là Product Details và Products; Stack còn Home.

6. Xem phần tử trên cùng bằng Peek()

```
string currentPage = pages.Peek();
```

Peek() chỉ xem phần tử trên cùng, không xóa phần tử.

```
Stack<string> pages =
    new Stack<string>();

pages.Push("Home");
pages.Push("Products");

Console.WriteLine(pages.Peek());
Console.WriteLine(pages.Peek());
```

Cả hai lần đều in Products và Stack vẫn giữ nguyên hai phần tử.

Method	Lấy phần tử trên cùng	Xóa phần tử
Peek()	Có	Không
Pop()	Có	Có

7. Lỗi khi Stack rỗng

```
Stack<string> pages =
    new Stack<string>();

pages.Pop(); // lỗi
pages.Peek(); // lỗi
```

Cả Pop() và Peek() đều ném InvalidOperationException khi Stack rỗng.

```
if (pages.Count > 0)
{
    string page = pages.Pop();
}
else
{
    Console.WriteLine("The stack is empty.");
}
```

Tuy nhiên, .NET có các method an toàn hơn: TryPop() và TryPeek().

8. TryPop() - lấy và xóa an toàn

```
bool result =
    pages.TryPop(out string? page);
```

- Nếu Stack có phần tử: trả về true, gán phần tử trên cùng vào page và xóa phần tử đó.
- Nếu Stack rỗng: trả về false, page nhận giá trị mặc định và không phát sinh exception.

```
if (pages.TryPop(out string? page))
{
    Console.WriteLine($"Back to: {page}");
}
else
{
    Console.WriteLine("There is no page.");
}
```

9. TryPeek() - xem phần tử an toàn

```
if (pages.TryPeek(out string? page))
{
    Console.WriteLine($"Current top page: {page}");
}
else
{
    Console.WriteLine("The stack is empty.");
}
```

TryPeek() xem phần tử trên cùng, không xóa và trả về false nếu Stack rỗng.

Method	Có xóa?	Khi Stack rỗng
Pop()	Có	Ném exception
Peek()	Không	Ném exception
TryPop()	Có	Trả về false
TryPeek()	Không	Trả về false

10. Thuộc tính Count

```
int numberOfPages = pages.Count;

Console.WriteLine(
    $"History count: {pages.Count}");
```

Nếu Stack chứa ba phần tử thì Count bằng 3.

11. Duyệt Stack bằng foreach

```
foreach (string page in pages)
{
    Console.WriteLine(page);
}
```

Stack được duyệt từ phần tử trên cùng xuống dưới. Với Home, Products, Details, kết quả thường là Details, Products, Home.

```
pages.Push("Home");
pages.Push("Products");
pages.Push("Details");

foreach (string page in pages)
{
    Console.WriteLine(page);
}
```

foreach chỉ đọc, không xóa phần tử. Sau vòng lặp, Count vẫn bằng 3.

12. Không thay đổi Stack trong foreach

Không nên Pop() trong lúc foreach đang duyệt vì collection đang bị thay đổi.

```
foreach (string page in pages)
{
    pages.Pop(); // không nên
}
```

Muốn xử lý cho đến khi Stack rỗng, dùng while với TryPop():

```
while (pages.TryPop(out string? page))
{
    Console.WriteLine(page);
}
```

Khi còn phần tử, TryPop() trả về true; khi hết phần tử, trả về false và vòng lặp kết thúc.

13. Contains() và Clear()

```
bool exists =
    pages.Contains("Home");
```

Contains() kiểm tra phần tử có tồn tại và không làm thay đổi Stack.

```
if (pages.Contains("Products"))
{
    Console.WriteLine(
        "Products page exists in history.");
}
```

Clear() xóa toàn bộ phần tử nhưng Stack vẫn tồn tại và có thể Push() tiếp.

```
pages.Clear();

// pages.Count == 0
pages.Push("Login");
```

14. Stack không có index

```
pages[0]; // không hợp lệ
```

Stack<T> không được thiết kế để truy cập ngẫu nhiên theo vị trí. Nó tập trung vào thao tác với phần tử trên cùng: thêm, xem và lấy phần tử trên cùng. Khi cần index, dùng List<T>.

15. Khởi tạo Stack có dữ liệu

```
Stack<string> pages =
    new Stack<string>(
        new[]
        {
            "Home",
            "Products",
            "Details"
        });
```

Cách truyền collection vào constructor có thể dễ gây nhầm về thứ tự đỉnh Stack. Khi mới học, nên Push() lần lượt để biết rõ phần tử nào được thêm sau cùng.

```
Stack<string> pages =
    new Stack<string>();

pages.Push("Home");
pages.Push("Products");
pages.Push("Details");
```

16. Encapsulation với Stack<T>

Nguyên tắc đóng gói giống List, Dictionary, HashSet và Queue: không để collection nội bộ là public.

```
public Stack<string> History { get; } // không nên
```

Nếu public, bên ngoài có thể Clear(), Push() dữ liệu rỗng hoặc Pop() tùy ý, làm class mất quyền kiểm soát.

```
private readonly Stack<string> _history;
```

Sau đó cung cấp các method có kiểm soát:

- VisitPage()
- ViewCurrentPage()
- GoBack()
- GetHistoryCount()
- DisplayHistory()

Ý nghĩa của private readonly: private ngăn truy cập trực tiếp từ bên ngoài; readonly ngăn field trở sang Stack khác sau constructor nhưng vẫn cho phép Push/Pop bên trong class.

17. Ví dụ hoàn chỉnh - BrowserHistory

```

public class BrowserHistory
{
    public string BrowserName { get; }

    private readonly Stack<string> _history;

    public BrowserHistory(string browserName)
    {
        if (string.IsNullOrEmpty(browserName))
        {
            throw new ArgumentException(
                "Browser name cannot be null or whitespace.",
                nameof(browserName));
        }

        BrowserName = browserName.Trim();
        _history = new Stack<string>();
    }

    public void VisitPage(string page)
    {
        if (string.IsNullOrEmpty(page))
        {
            throw new ArgumentException(
                "Page cannot be null or whitespace.",
                nameof(page));
        }

        _history.Push(page.Trim());
    }

    public string? ViewCurrentPage()
    {
        if (_history.TryPeek(out string? page))
        {
            return page;
        }

        return null;
    }

    public string? GoBack()
    {
        if (_history.TryPop(out string? page))
        {
            return page;
        }

        return null;
    }

    public int GetHistoryCount()
    {
        return _history.Count;
    }

    public void DisplayHistory()
    {
        Console.WriteLine(
            $"Browser history: {BrowserName}");

        if (_history.Count == 0)
        {
            Console.WriteLine(
                "The browsing history is empty.");
            return;
        }

        foreach (string page in _history)
        {
            Console.WriteLine($"- {page}");
        }
    }
}

```

18. Hiểu đúng GoBack()

GoBack() trong ví dụ sử dụng TryPop(), vì vậy nó trả về trang vừa bị lấy khỏi Stack, không phải trang mới hiện tại sau khi quay lại.

```
Details ← trên cùng  
Products  
Home  
  
string? removedPage = history.GoBack();
```

Kết quả: removedPage bằng Details; Stack còn Products và Home. Trang hiện tại mới là Products.

```
history.GoBack();  
  
string? currentPage =  
    history.ViewCurrentPage();
```

Điểm dễ nhầm: TryPop() trả về phần tử vừa bị lấy ra; TryPeek() trả về phần tử hiện đang ở trên cùng.

19. Program.cs minh họa

```

public class Program
{
    public static void Main(string[] args)
    {
        try
        {
            BrowserHistory history =
                new BrowserHistory("My Browser");

            history.VisitPage("Home");
            history.VisitPage("Products");
            history.VisitPage("Product Details");

            Console.WriteLine(
                "=== BROWSER HISTORY ===");

            history.DisplayHistory();

            Console.WriteLine(
                "\n=== CURRENT PAGE ===");

            string? currentPage =
                history.ViewCurrentPage();

            if (currentPage == null)
            {
                Console.WriteLine(
                    "There is no current page.");
            }
            else
            {
                Console.WriteLine(
                    $"Current page: {currentPage}");
            }

            Console.WriteLine(
                "\n=== GO BACK ===");

            string? removedPage =
                history.GoBack();

            if (removedPage == null)
            {
                Console.WriteLine(
                    "There is no page to remove.");
            }
            else
            {
                Console.WriteLine(
                    $"Leaving page: {removedPage}");
            }

            string? pageAfterGoingBack =
                history.ViewCurrentPage();

            if (pageAfterGoingBack == null)
            {
                Console.WriteLine(
                    "There is no current page.");
            }
            else
            {
                Console.WriteLine(
                    $"Current page: {pageAfterGoingBack}");
            }

            Console.WriteLine(
                $"History count: " +
                $"{history.GetHistoryCount()}");

            history.DisplayHistory();
        }
        catch (ArgumentException ex)
        {

```

```

        Console.WriteLine(
            $"Invalid argument: {ex.Message}");
    }
    catch (Exception ex)
    {
        Console.WriteLine(
            $"Unexpected error: {ex.Message}");
    }
}
}

```

20. Kiểm tra TryPeek() không giảm Count

```

int countBefore =
    history.GetHistoryCount();

string? currentPage =
    history.ViewCurrentPage();

int countAfter =
    history.GetHistoryCount();
if (countBefore == countAfter)
{
    Console.WriteLine(
        "TryPeek did not change the Stack.");
}

```

Kết quả mong đợi: Count trước bằng 3 và Count sau vẫn bằng 3.

21. Kiểm tra TryPop() giảm Count đúng một

```

int countBefore =
    history.GetHistoryCount();

string? page =
    history.GoBack();

int countAfter =
    history.GetHistoryCount();
if (page != null &&
    countAfter == countBefore - 1)
{
    Console.WriteLine(
        "TryPop removed exactly one page.");
}

```

22. Xử lý đến khi Stack rỗng

Có thể dùng Count:

```

while (history.GetHistoryCount() > 0)
{
    string? page = history.GoBack();

    if (page != null)
    {
        Console.WriteLine(
            $"Removed page: {page}");
    }
}

```

Hoặc dựa trực tiếp vào kết quả nullable:

```
while (true)
{
    string? page = history.GoBack();

    if (page == null)
    {
        break;
    }

    Console.WriteLine(
        $"Removed page: {page}");
}
```

Cách thứ hai tận dụng trực tiếp kết quả null để kết thúc vòng lặp.

23. Ứng dụng của Stack trong .NET

23.1 Undo thao tác

Các thao tác gần nhất như nhập văn bản, xóa văn bản hoặc đổi màu sẽ được hoàn tác trước.

23.2 Lịch sử trang

Trang truy cập gần nhất được quay lại trước.

23.3 Xử lý biểu thức

- Kiểm tra dấu ngoặc.
- Tính biểu thức.
- Chuyển đổi biểu thức trung tố sang hậu tố.

23.4 Hệ quy và call stack

Khi method gọi method khác, runtime dùng call stack để lưu vị trí đang thực hiện, biến cục bộ và địa chỉ quay lại.

23.5 Duyệt cây và đồ thị

Thuật toán DFS thường dùng Stack.

23.6 Redo

Hệ thống Undo/Redo thường dùng hai Stack: Undo Stack và Redo Stack. Phần này sẽ học nâng cao sau.

24. So sánh Queue và Stack

Đặc điểm	Queue<T>	Stack<T>
Nguyên tắc	FIFO	LIFO
Thêm	Enqueue()	Push()
Lấy và xóa	Dequeue()	Pop()
Xem	Peek()	Peek()
Lấy an toàn	TryDequeue()	TryPop()
Xem an toàn	TryPeek()	TryPeek()
Phần tử được lấy	Phần tử vào trước	Phần tử vào sau

Queue:
A vào trước
B vào sau
→ A ra trước

Stack:
A vào trước
B vào sau
→ B ra trước

25. So sánh các collection đã học

Collection	Đặc điểm chính	Thao tác đặc trưng
List<T>	Danh sách có index	Add, Remove, [index]
Dictionary<TKey,TValue>	Key - Value	TryGetValue, Remove(key)
HashSet<T>	Không chứa phần tử trùng	Add, Contains, Remove
Queue<T>	FIFO	Enqueue, TryDequeue
Stack<T>	LIFO	Push, TryPop

26. Các lỗi thường gặp

26.1 Gọi Pop() khi Stack rỗng

```
_stack.Pop(); // có thể ném exception
_stack.TryPop(out string? item); // an toàn hơn
```

26.2 Nhầm Peek() với Pop()

Peek
→ chỉ xem

Pop
→ xem và xóa

26.3 Không kiểm tra nullable

```
history.GoBack().ToUpper(); // không nên
string? page = history.GoBack();

if (page != null)
{
    Console.WriteLine(page);
}
```

26.4 Dùng điều kiện vòng lặp ngược

```
while (stack.Count == 0) // sai logic
{
    stack.TryPop(out string? item);
}
while (stack.Count > 0)
{
    stack.TryPop(out string? item);
}
while (stack.TryPop(out string? item))
{
    // xử lý item
}
```

26.5 Thay đổi Stack trong foreach

```
foreach (string item in stack)
{
    stack.Pop(); // không nên
}
```

26.6 Mong Stack có index

```
stack[0]; // không hợp lệ
```

26.7 Để Stack là public

```
public Stack<string> History { get; } // không nên
private readonly Stack<string> _history; // nên dùng
```

27. Tóm tắt cần nhớ

Bản chất: Stack<T> là ngăn xếp LIFO: vào sau, ra trước và không có index.

Member	Ý nghĩa
Push(item)	Thêm phần tử lên đỉnh Stack
Pop()	Lấy và xóa phần tử trên cùng; lỗi nếu Stack rỗng
Peek()	Xem phần tử trên cùng; không xóa; lỗi nếu Stack rỗng
TryPop(out item)	Lấy và xóa an toàn; trả về bool
TryPeek(out item)	Xem an toàn; trả về bool
Count	Số phần tử
Contains(item)	Kiểm tra tồn tại
Clear()	Xóa toàn bộ

28. Bài tập 8.5 - Quản lý lịch sử thao tác

Tạo class ActionHistory.

28.1 Thuộc tính và collection

```
public string Name { get; }
private readonly Stack<string> _actions;
```

28.2 Method 1 - AddAction()

```
public void AddAction(string action)
```

- action không được rỗng.
- Dùng Trim().
- Dùng Push().

28.3 Method 2 - ViewLastAction()

```
public string? ViewLastAction()
```

- Dùng TryPeek().

- Trả về thao tác gần nhất.
- Stack rỗng thì trả về null.

28.4 Method 3 - UndoLastAction()

```
public string? UndoLastAction()
```

- Dùng TryPop().
- Trả về thao tác vừa bị hoàn tác.
- Stack rỗng thì trả về null.

28.5 Method 4 - GetActionCount()

```
public int GetActionCount()
```

Trả về Count.

28.6 Method 5 - DisplayActions()

```
public void DisplayActions()
```

- Hiển thị Name.
- Kiểm tra Stack rỗng.
- Duyệt bằng foreach.
- Không làm thay đổi Stack.

29. Yêu cầu trong Program.cs

3. Tạo một ActionHistory.
4. Thêm ít nhất ba thao tác.
5. Hiển thị toàn bộ lịch sử.
6. Xem thao tác gần nhất bằng ViewLastAction().
7. Kiểm tra null trước khi hiển thị.
8. Xác nhận TryPeek() không làm giảm Count.
9. Hoàn tác một thao tác bằng UndoLastAction().
10. Xác nhận TryPop() làm Count giảm đúng một.
11. Hiển thị lịch sử sau khi hoàn tác.
12. Tiếp tục hoàn tác cho đến khi Stack rỗng.
13. Thử hoàn tác thêm một lần khi Stack đã rỗng.
14. Hiển thị thông báo phù hợp.

Bắt buộc sử dụng: Stack<string>, Push(), TryPeek(), TryPop(), Count và foreach. Chưa cần làm chức năng Redo.

30. Checklist trước khi nộp bài

- Stack được khai báo private readonly.
- Chuỗi được kiểm tra bằng string.IsNullOrEmpty() và Trim().
- ViewLastAction() trả về string?.
- UndoLastAction() trả về string?.
- Không gọi method trực tiếp trên giá trị có thể null.
- TryPeek() không làm Count thay đổi.
- TryPop() làm Count giảm đúng một khi thành công.
- Không Pop() trong foreach.
- Vòng lặp xử lý đến khi Stack rỗng có điều kiện đúng.
- Khi Stack rỗng, chương trình hiển thị thông báo phù hợp và không ném lỗi ngoài ý muốn.

Hết Phần 8.5 - Stack<T>.